

732A94 Advanced R programming

Lecture 5

Johan Alenlöv (Slides based on Leif Jonssons, Måns Magnusson and Krzysztof Bartoszek)

Linköpings Universitet

- Input and output
- Basic I/O
- Cloud storage
- Web APIs (Lab!)
- Web scraping
- Shiny
- Relational databases

Questions?

Any questions from last time?

Two questions decide almost everything:

Format – how is the data encoded?

Localization – where does the data live?

<http://www.joelonsoftware.com/articles/Unicode.html>

The Absolute Minimum Every Software Developer Absolutely, Positively Must Know About Unicode and Character Sets (No Excuses!)

- Unicode defines codes for all (?) characters – but there are multiple *encodings* (for a given language only a small fraction of the characters are used)
- HTML has a **Content-Type** tag
- BUT: e-mail, **.txt**, **.csv** do not

Encoding in R

```
x <- "Linköping"  
Encoding(x)
```

```
## [1] "UTF-8"
```

```
iconv(x, from = "UTF-8", to = "ASCII", sub = "?")
```

```
## [1] "Link??ping"
```

```
Sys.getlocale("LC_CTYPE")
```

```
## [1] "en_US.UTF-8"
```

`read.csv(..., encoding = "latin1")` and `readr::read_csv(..., locale = locale(encoding = "latin1"))` are how you tell R what it is reading.

“Formats”

Plain text

- `.txt`, `.csv`, `.tsv`, fixed width

Structured text

- JSON, XML, YAML, HTML

Binary

- `.RData` / `.rds`, `.xlsx`, `.sav`, Parquet, images

There is no such thing as “just a file” – you always need to know which of these you have.

	Local	Remote
Arbitrary data	Own computer	Cloud storage
	Local network	Web pages, scraping
Structured	Local database	Web APIs
		Remote database

Files on your computer

```
# Input simple data
read.table(); read.csv(); read.csv2()
read.csv("clipboard")
load()           # .RData
readRDS()        # a single object

# Output simple data
write.table(); write.csv(); write.csv2()
write.csv(x, "clipboard")
save()           # .RData
saveRDS()        # a single object
```

`read.csv2()/write.csv2()` use `;` as separator and `,` as decimal mark – the Swedish/European convention.

Faster and stricter alternatives

```
readr::read_csv("data.csv")           # tibble, guesses types  
data.table::fread("data.csv")         # very fast  
arrow::read_parquet("data.parquet") # columnar, typed
```

`read.csv()` silently guesses; `readr` tells you what it guessed and lets you specify `col_types`. For big files that matters a lot.

Software / data	R package
Excel	<code>readxl</code> , <code>openxlsx</code>
SAS, SPSS, Stata	<code>haven</code> (or <code>foreign</code>)
XML	<code>xml2</code>
JSON, GeoJSON	<code>jsonlite</code>
Text documents	<code>tm</code> , <code>quanteda</code>
Maps / spatial	<code>sf</code>
Images / rasters	<code>terra</code> , <code>magick</code>

Format issues: data stored in column names

```
m <- cbind("id" = c("A", "B"),  
           "2020" = c(1, 2), "2021" = c(5, 6))  
m
```

```
##      id  2020 2021  
## [1,] "A"  "1"  "5"  
## [2,] "B"  "2"  "6"
```

The years are *data*, but they are stored as column names – and note that `cbind` turned the numbers into strings.

Format issues: fixing it

```
library(tidyr)
d <- data.frame(id = c("A", "B"),
                `2020` = c(1, 2), `2021` = c(5, 6),
                check.names = FALSE)
pivot_longer(d, -id, names_to = "year",
             values_to = "value")
```

```
## # A tibble: 4 x 3
##   id      year value
##   <chr> <chr> <dbl>
## 1 A      2020     1
## 2 A      2021     5
## 3 B      2020     2
## 4 B      2021     6
```

More on tidy data in the bonus lecture.

Cloud storage: why?

- Robust
- Backups
- Cloud computing

... can be tricky in the beginning

... but how about safety? (data leaks, outsourcing)

... and control over what is going on? (outsourcing, denial of service)

BUT it requires an internet connection.

Remote	Package
General	<code>curl</code> , <code>httr2</code>
GitHub	<code>gh</code> , <code>gitcreds</code>
Dropbox	<code>rdrop2</code>
Amazon S3	<code>paws</code> , <code>aws.s3</code>
Google Drive/Sheets	<code>googledrive</code> , <code>googlesheets4</code>

Never put credentials in your code. Use `.Renviron`,
`Sys.getenv("MY_API_KEY")` and add `.Renviron` to `.gitignore`.

An application programming interface using http.

A “contract to *get data* online”.

More and more common. Examples:

- GitHub
- Riksdagen
- Statistics Sweden (SCB), **pxweb**

Basic principles:

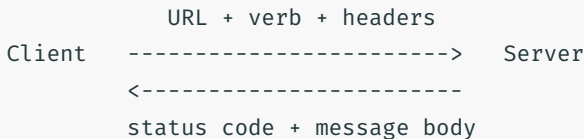
- Data is returned (JSON / XML)
- Each specific piece of data has its own URI
- Communication is based on HTTP verbs

Anatomy of a URL

```
http://www.domain.com:1234/path/to/resource?a=b&x=y
\____/ \_____/ \___/ \_____/ \_____/
protocol host      port  resource path  query
```

The **query string** after ? is how you pass parameters – ?a=b&x=y is two parameters, a and x.

Hypertext Transfer Protocol (http)



1. The client sends a **request**: a URL, a verb and some headers
2. The server sends a **response**: a status code and a body

Stateless: the server remembers nothing between requests.

Verb	Description
GET	Get “data” from the server
POST	Post “data” to the server (to get something)
PUT	Update “data” on the server
DELETE	Delete a resource on the server

For data analysis you will almost always use **GET**.

Code	Description
1XX	Information from the server
2XX	Yay! Gimme' data!
3XX	Redirections
4XX	You failed
5XX	The server failed

200 OK, 404 not found, 403 forbidden, 429 too many requests, 500 internal server error.

Air quality in Sweden:

<https://www.naturvardsverket.se/amnesomraden/luft/statistik--utslapp-och-halter/>

Google Maps Geocoding API:

<https://developers.google.com/maps/documentation/geocoding/intro>

Statistics Sweden through the **pxweb** package.

```
library(httr2)
resp <- request("https://api.github.com") |>
  req_url_path("repos", "STIMALiU", "AdvRCourse") |>
  req_user_agent("732A94 course example") |>
  req_perform()

resp_status(resp)           # 200 we hope
dat <- resp_body_json(resp) # a named list
dat$full_name
```

Always check the status code before you trust the body – that is “check your input” for web data.

JavaScript Object Notation (JSON)

- Think of named lists in R
- R packages: `jsonlite`

Extensible Markup Language (XML)

- Older format (using nodes)
- `xpath` to select nodes
- R packages: `xml2`

```
{
  "firstName": "John",
  "lastName": "Smith",
  "age": 25,
  "address": {
    "streetAddress": "21 2nd Street",
    "city": "New York"
  },
  "phoneNumber": [
    { "type": "home", "number": "212 555" },
    { "type": "fax", "number": "646 555" }
  ],
  "newSubscription": false,
  "companyName": null
}
```

```
library(jsonlite)
js <- '{"firstName":"John","age":25,
      "address":{"city":"New York"}}'
p <- fromJSON(js, simplifyVector = FALSE)
p$address$city
```

```
## [1] "New York"
```

```
toJSON(list(a = 1:3, b = "x"), auto_unbox = TRUE)
```

```
## {"a":[1,2,3],"b":"x"}
```

`simplifyVector = TRUE` (the default) tries to turn JSON arrays into vectors and data frames – convenient, but sometimes surprising.

```
<?xml version="1.0" encoding="utf-8"?>
<wikimedia>
  <projects>
    <project name="Wikipedia" launch="2001-01-05">
      <editions>
        <edition language="English">en.wikipedia.org</edition>
        <edition language="German">de.wikipedia.org</edition>
        <edition language="Polish">pl.wikipedia.org</edition>
      </editions>
    </project>
  </projects>
</wikimedia>
```

```
library(xml2)
x <- read_xml(
  '<wikimedia><projects><project name="Wikipedia">
    <editions>
      <edition language="English">en.wikipedia.org</edition>
      <edition language="German">de.wikipedia.org</edition>
    </editions></project></projects></wikimedia>')
xml_text(xml_find_all(x, "//edition"))

## [1] "en.wikipedia.org" "de.wikipedia.org"

xml_attr(xml_find_all(x, "//edition"), "language")

## [1] "English" "German"
```

- Unstructured http(s) data
- Often HTML format
- Spiders / scraping / web crawlers
- The basics behind search engines

Use an API if there is one. Scrape only when there is not.

```
<!DOCTYPE html>
<html>
  <head>
    <title>This is a title</title>
  </head>
  <body>
    <p class="greeting">Hello world!</p>
  </body>
</html>
```

HTML is a *tree*, so the same `xpath` ideas apply – plus CSS selectors, which are usually easier: `p.greeting`, `#id`, `table tr td`.

Simplifies spider activity:

- Download data
- Parse data
- Follow links
- Fill out forms
- Store crawling history

```
library(rvest)
page <- minimal_html(
  '<html><body><h1>A title</h1>
  <p class="a">One</p><p class="a">Two</p>
  </body></html>')
html_text2(html_element(page, "h1"))
```

```
## [1] "A title"
```

```
html_text2(html_elements(page, "p.a"))
```

```
## [1] "One" "Two"
```

For a real page: `read_html("https://...")` then the same functions.

`html_table()` pulls out an entire HTML table as a data frame.

Scraping is fragile! The page layout changes and your code breaks silently.

- `www.domain.se/robots.txt` says what you may crawl
- Politeness: identify yourself, rate limit, cache what you download (`polite`, `httr2::req_throttle()`)
- Robot traps
- JavaScript-rendered pages (nothing in the HTML you download)
- Delays

Check the terms of service. Scraping personal data has GDPR implications.

- Shiny is an R package that makes it easy to build web apps (both local and internet based) using R
- A major use case: interactive dashboards for data visualisation/analysis in commercial and research applications
- Shiny lets users without advanced web-development skills build apps with user interfaces that are **reactive** to user actions; in essence Shiny generates HTML from R
- Advanced web developers can extend Shiny apps with CSS themes, `htmlwidgets` and JavaScript actions; `HTML()` adds your own HTML to the UI

Building a basic Shiny app: I

- Each Shiny app must have two components – a **ui** object (the frontend) and a **server** function (the backend)
- The UI is written as a layout function, customised with one or more *input* and *output* functions
- The server function performs all the data access, possibly using the inputs obtained through the UI, and assigns everything to be displayed to **output**, wrapped in **render*** functions
- The server function can take up to three parameters: **input**, **output** and **session** (for managing multiple user sessions)

- The UI and server components can be written in a single `.R` file or in two separate files
- The app object is created when the UI and server are passed to `shiny::shinyApp()` or `shiny::runGitHub()`
- **Important:** when writing a Shiny app as an R package, store the `.R` files with the ui/server components in the root package directory or in the package's `inst/` folder
- Tip: when subsetting data, especially data with missing values, consider `subset()` or `dplyr::filter()`

A simple example

```
library(shiny)
ui <- fluidPage(
  numericInput(inputId = "n", label = "How many tosses?",
    value = 4, min = 1, max = 100, step = 1),
  plotOutput(outputId = "probs")
)
server <- function(input, output) {
  output$probs <- renderPlot({
    barplot(dbinom(0:input$n, input$n, 0.5),
      names = 0:input$n, ylab = "Probability")
  })
}
shinyApp(ui, server)
```

See 732A94_AdvancedRHT2026_Lecture05_ShinyAppExample.R

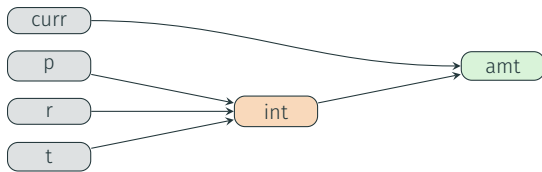
A simple example: what to notice

- `numericInput` is an example of **input validation**: app users can only enter integers between 1 and 100
- The `inputId` in the UI is what the server reads as `input$n`
- The `outputId` in the UI is what the server writes as `output$probs`
- App objects generated by `shinyApp(ui, server)` open in a web browser by default

Reactivity (reactive programming): I

- Shiny lets us build **reactive** apps, where outputs change whenever a user manipulates the UI to change an input
- For reactivity the server function must always access UI inputs inside a **reactive context** created by constructs such as **render*** functions or **reactive()**
- Content assigned to a UI output must likewise be wrapped in a reactive context
- Internally Shiny builds a **reactive graph** capturing the dependencies between input and output objects, and executes them lazily

Reactivity: the reactive graph



`curr`, `p`, `r`, `t` are inputs Shiny listens to; `int` is an intermediate reactive; `amt` is the output that connects back to the ui.

```
server <- function(input, output) {  
  int <- reactive(input$p * input$r * input$t / 100)  
  output$amt <- renderText({  
    paste("Amount at maturity:", input$curr,  
          round(input$p + int(), 2))  
  })  
}
```

`int` is computed once per change and re-used – that is why you want `reactive()` rather than repeating the expression.

See `732A94_AdvancedRHT2026_Lecture05_ReactiveAppExample.R`

- Locally
- Zip-file in the cloud
- GitHub (see `runGitHub()`)
- Your own server (Shiny Server / Posit Connect)
- `shinyapps.io`

- Structured database in tables
- Local or online
- Query language for I/O (SQL)
- Effective for big data
- Difficult to design

The relational model

A_1	...	A_n
value		← tuple (row), unordered

- **Relation** (table): the whole thing, named by a relation variable R
- **Attribute** (column): $A_1, \dots, A_n$, unordered
- **Heading**: the attribute names; **body**: the set of tuples
- **Tuple** (row): one observation, unordered

Superkey: a “set of attributes such that two distinct rows do not have the same values for these attributes”

Candidate key: a minimal superkey

Primary key: the chosen candidate key – relationships between tables are made through the primary key

Foreign key: an attribute in one table that references the primary key of another

<https://en.wikipedia.org/wiki/Superkey>

Can be difficult to design.

- No duplicates
- No redundancies
- Easy to update
- “Normal forms”
- Easy to query

Database normalization “is the process of restructuring a relational database in accordance with a series of so-called normal forms in order to reduce data redundancy and improve data integrity” (usually: divide a table into separate tables linked by primary keys).

Denormalization: create redundancies for increased performance

- (preferred) store normalized data and let the DBMS create the additional redundancies – the DBMS is responsible for inconsistencies
- (common) design a denormalized DB – the designer is responsible for inconsistencies

First normal form: in each attribute (column) entry there is a single atomic value – for Telephone Number you cannot have two telephone numbers.

Second normal form: 1NF, and each non-prime attribute depends functionally only on the whole key, not on part of it.

(Course code, Course name, University, University country) is not in 2NF: University country is determined by University. Here (Course code, University) is the composite primary key.

Third normal form: 2NF and “every non-prime attribute of R is non-transitively dependent on every key of R ”.

(University, Year, Vice-Chancellor, Vice-Chancellor DOB) with composite key
(University, Year): Vice-Chancellor DOB depends on the key *via* Vice-Chancellor.
What if someone makes a typo the second time they enter it?

Boyce–Codd normal form (3.5NF): stricter than 3NF, no functional dependencies between two attributes of which neither is a superkey.

(city, land plot, postal code) fails, because of the relationship between city and postal code.

Fourth normal form: 3NF and no multiple multivalued dependencies.

(Teacher, Language, Course), primary key is the whole entry:

Version 1 (redundant)	Version 2 (what if I stop teaching R?)
KB, Polish, 732A94	KB, Polish, 732A94
KB, Polish, 732A63	KB, English, 732A94
KB, English, 732A94	KB, Swedish, 732A63
KB, English, 732A63	
KB, Swedish, 732A94	
KB, Swedish, 732A63	

Fifth normal form: when there are complex constraints on the possible combinations of values.

Sixth normal form: when there are temporal dependencies in the data (can lead to table explosion).

Domain-key normal form: values are only constrained by the permissible values for the attributes and by the key uniquely identifying the row. (Lecturer, Lecturer description, University) fails – the description repeats the university.

Database system	R package
SQLite (file / in-memory)	RSQLite
PostgreSQL	RPostgres
MySQL / MariaDB	RMariaDB
ODBC (Access, SQL Server)	odbc
Oracle	ROracle
DuckDB (analytics)	duckdb
MongoDB (not relational)	mongolite

They all implement the common **DBI** interface.

```
library(DBI)
con <- dbConnect(RSQLite::SQLite(), ":memory:")
dbWriteTable(con, "iris", iris)
dbListTables(con)
```

```
## [1] "iris"
```

```
dbGetQuery(con, 'SELECT Species, AVG("Sepal.Length") AS m
                  FROM iris GROUP BY Species')
```

```
##      Species      m
## 1    setosa 5.006
## 2 versicolor 5.936
## 3  virginica 6.588
```

Note the double quotes around "Sepal.Length": it contains a dot, which SQL would otherwise read as *table.column*.

```
dbGetQuery(con, 'SELECT COUNT(*) AS n FROM iris
                WHERE Species = ? AND "Sepal.Length" > ?',
            params = list("setosa", 5))
```

```
##      n
```

```
## 1 22
```

Use parameters (?) rather than pasting values into the SQL string – pasting is how SQL injection happens. It is also **check your input** in another disguise.

dplyr straight onto the database

```
library(dplyr)
q <- tbl(con, "iris") |>
  group_by(Species) |>
  summarise(m = mean(Sepal.Length, na.rm = TRUE))
show_query(q)           # see the generated SQL
```

```
## <SQL>
## SELECT `Species`, AVG(`Sepal.Length`) AS `m`
## FROM `iris`
## GROUP BY `Species`
```

dplyr straight onto the database

```
collect(q) # bring the result into R
```

```
## # A tibble: 3 x 2
##   Species      m
##   <chr>      <dbl>
## 1 setosa      5.01
## 2 versicolor  5.94
## 3 virginica   6.59
```

```
dbDisconnect(con)
```

`dbplyr` translates the pipeline into SQL, so the *database* does the work and only the result travels to R. Very useful when the table does not fit in memory.

ALWAYS CHECK INPUT!

Questions?

See you next time!